SFWRENG / MECHTRON 3K04 Software Development

This Week: Modeling the System

McMaster University
Engineering
Computing & Software

September 29th, 2026

Dr. Angela Zavaleta Bernuy

Requirements Engineering: Key Ideas

- Requirements engineering is about **eliciting, documenting, and validating what a system must do**; validation asks whether we have the *right requirements*, while verification asks whether we *built the system right*.
- Stakeholders include anyone who uses, is affected by, regulates, or approves the system, not just direct users.
- Requirements must account for **safety, security, reliability, performance, usability, accessibility, and traceability**, especially in safety-critical systems.
- Elicitation uses multiple techniques such as interviews, observation, workshops, document analysis, questionnaires, prototypes, and the **5 Whys** (because stakeholders often provide incomplete or conflicting information).
- Ethics and bias matter: requirements determine **who benefits, who carries risk, and what assumptions become part of the system**.

Writing and Testing Good Requirements

- **Functional requirements (FRs)** describe *what the system does*, and **Non-functional requirements (NFRs)** describe qualities, limits, or constraints.
- Good requirements should be **atomic, unambiguous, verifiable, complete, consistent, traceable, and implementation-free**.
- Requirements describe **WHAT** must be achieved, and not **HOW** it should be implemented.
- User stories capture the stakeholder perspective: **"As a [role], I want [goal], so that [benefit]."**
  - Strong stories follow **INVEST**: Independent, Negotiable, Valuable, Estimable, Small, Testable.
- Acceptance criteria define when a story is complete using **Given/When/Then**, with observable pass/fail outcomes and both normal and off-normal cases.
- Traceability connects each requirement back to its source and forward to the stories, design components, and tests that implement it.

Model-Driven Engineering (MDE)

- The idea of model-driven software development originated many years ago (used to be called *case tools*)
- However, only relatively recently has it become a reality
- Models have been in use for decades in engineering and science, so this is a natural progression
  - We have models for molecules, chemical compounds, etc.
  - Now, we have various ways to model software.

Model-Driven Engineering (MDE)

- It goes by other names such as Model-Based Design
- Main idea:
  - Models of the system you want to build, and its environments
  - Transformation of system models through various phases, until we can generate implementations (mechanical/electrical artifacts, code, or other logic implementation)
- Why?
  - Modelling is a form of abstraction:
    - It lets us get rid of some lower-level details in order to get a better understanding of the system
  - Impossible for anyone to understand every line of code in more complex systems:
    - Abstractions are necessary for development and documentation

Model-Driven Engineering (MDE): Benefits

- Many types of **analyses** can be performed on the models **to validate** the initial model
- **Correctness-by-construction** all the way through to implementation
  - Can also generate “proofs” or at least, checks, that correctness-by-construction is achieved
- Can construct **tools to help examine** the models
- Can **regenerate implementations** when changes are introduced
- **Low skill floor** and **high skill ceiling**
  - Easy to get started and allows for high amount of expression
- Facilitates **general reasoning and thinking** about software engineering and development

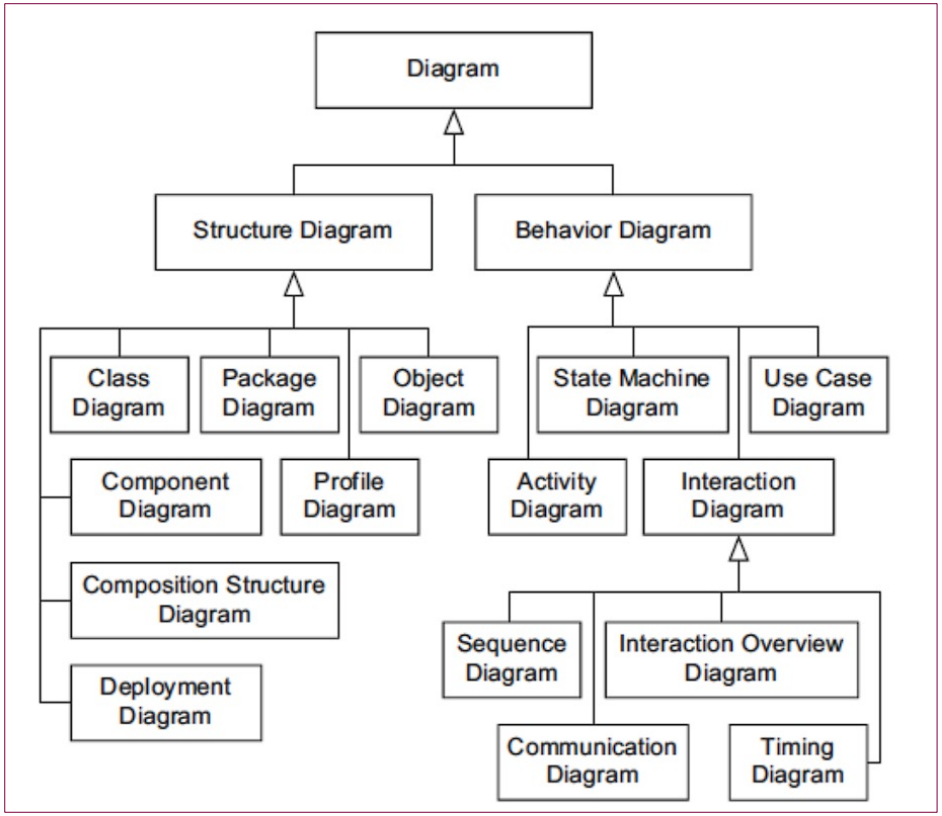
Correctness-by-Construction (CbC)

- Build programs incrementally based on a formal specification from the start
- Follows a specification-first, refinement-based approach
  - The specification is defined first. i.e. precondition and postcondition
  - Then, the program is successively created using a small set of refinement rules
    - For instance, let's say we are creating a program to find the larger value of an array
    - We first may start with an empty list and create refinement rules for that. Then, we we can define iteration rules.
    - We also need to verify that the loop maintains its properties (keeping the largest value).
  - These rules define side conditions preserving the correctness of the program
- Advantage: Errors are likely to be detected earlier in the design process and can be tracked more easily

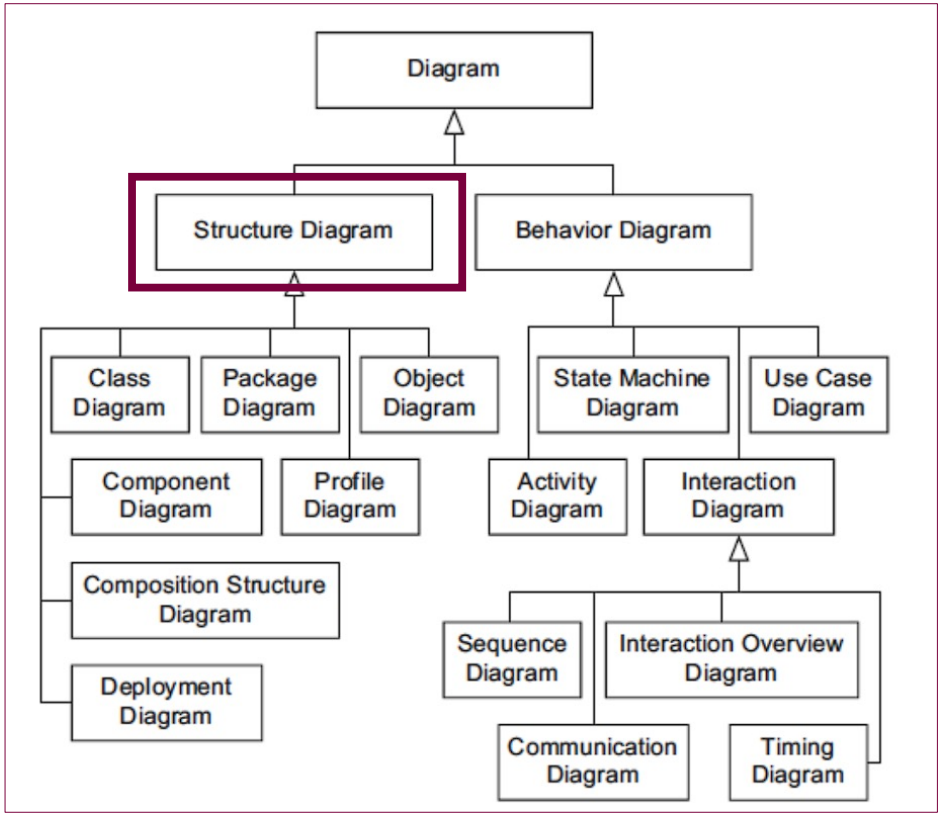
Model-Driven Engineering (MDE): Cons

- **Not a lot of tool support** compared to traditional forms of software engineering
- **Difficult** to implement **at scale**:
  - Requires common understanding across practitioners
- Loses benefits with more **simple systems**
  - Can be viewed as redundant by some
- Generally, **developers want to code not model**
  - … until it is too late

Unified Modeling Language (UML)

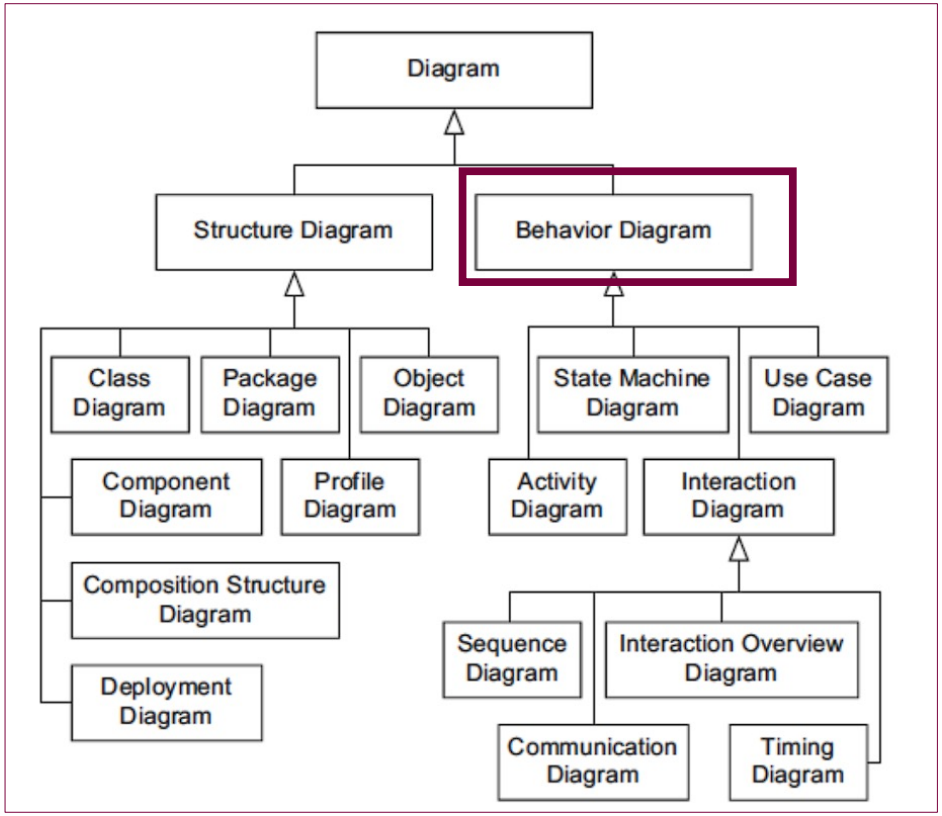


UML: Structure Diagram



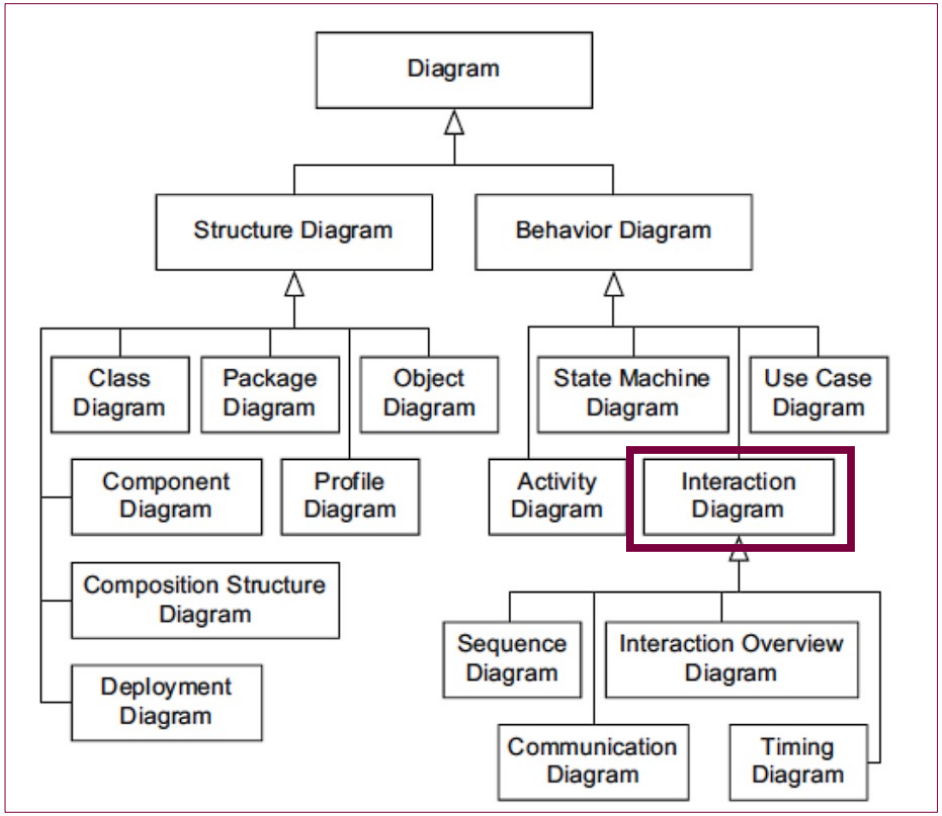
- Used primarily to determine how a system is glued together
- Often used to model system architectures, requirements, and defining domain specific languages (DSL)
- Best for defining modules/components and relationships between modules/components
- *Just like the blueprints of a building: they tell us what parts exist and how they fit together*

UML: Behaviour Diagram



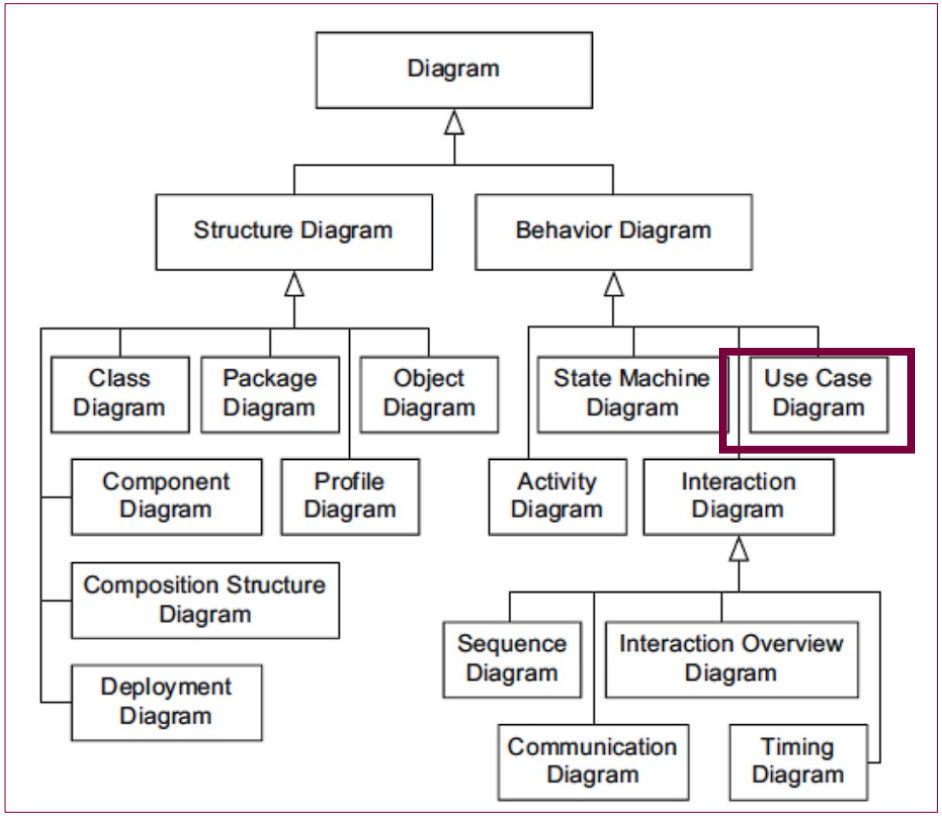
- Used primarily to describe what a system does, sometimes how it does it
- Often used for defining requirements, designs, and implementations
- Best for control theory and system development
- *If structure diagrams are the static blueprints, behaviour diagrams are the animations, they show how the system moves, reacts and communicates*

UML: Interaction Diagram

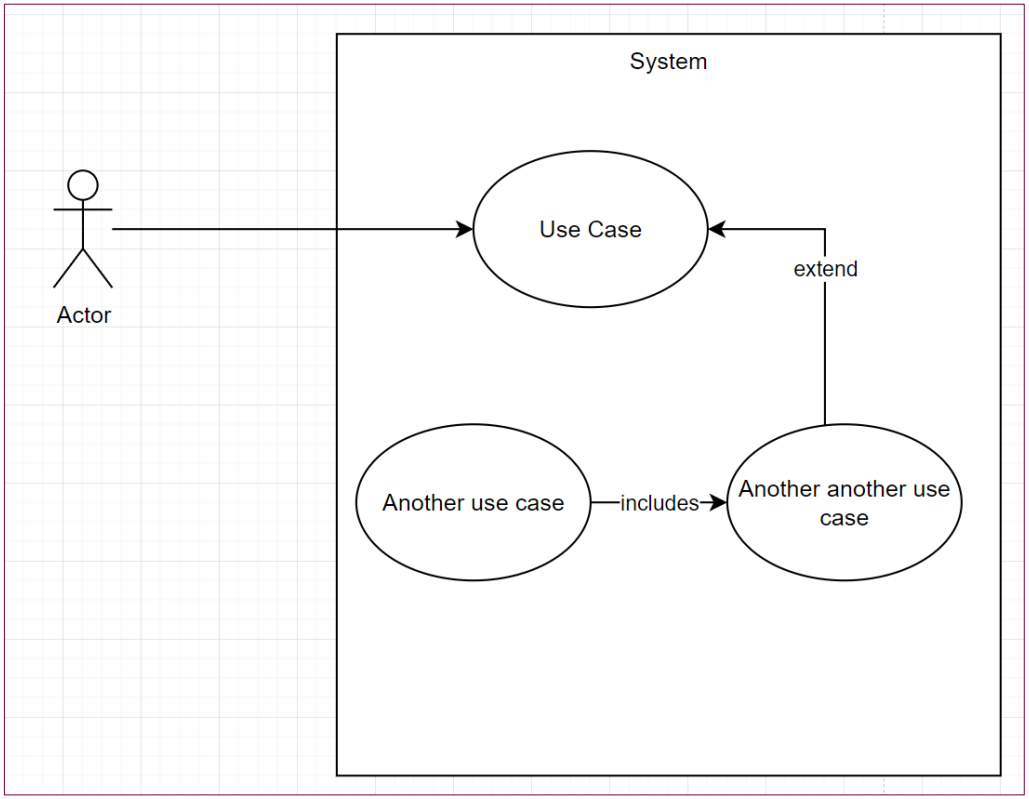


- A subset of Behaviour Diagrams specialized in capturing behavior between objects
- More niche in MDE: if more details are needed about specific behaviors that is when you might use interaction diagrams
- *Interaction diagrams zoom in on communication: they tell the story of how parts of the system exchange messages to make behavior happen. In MDE, these diagrams often feed into simulation and code generation steps.*

UML: Use Case Diagram



UML: Use Case Diagram

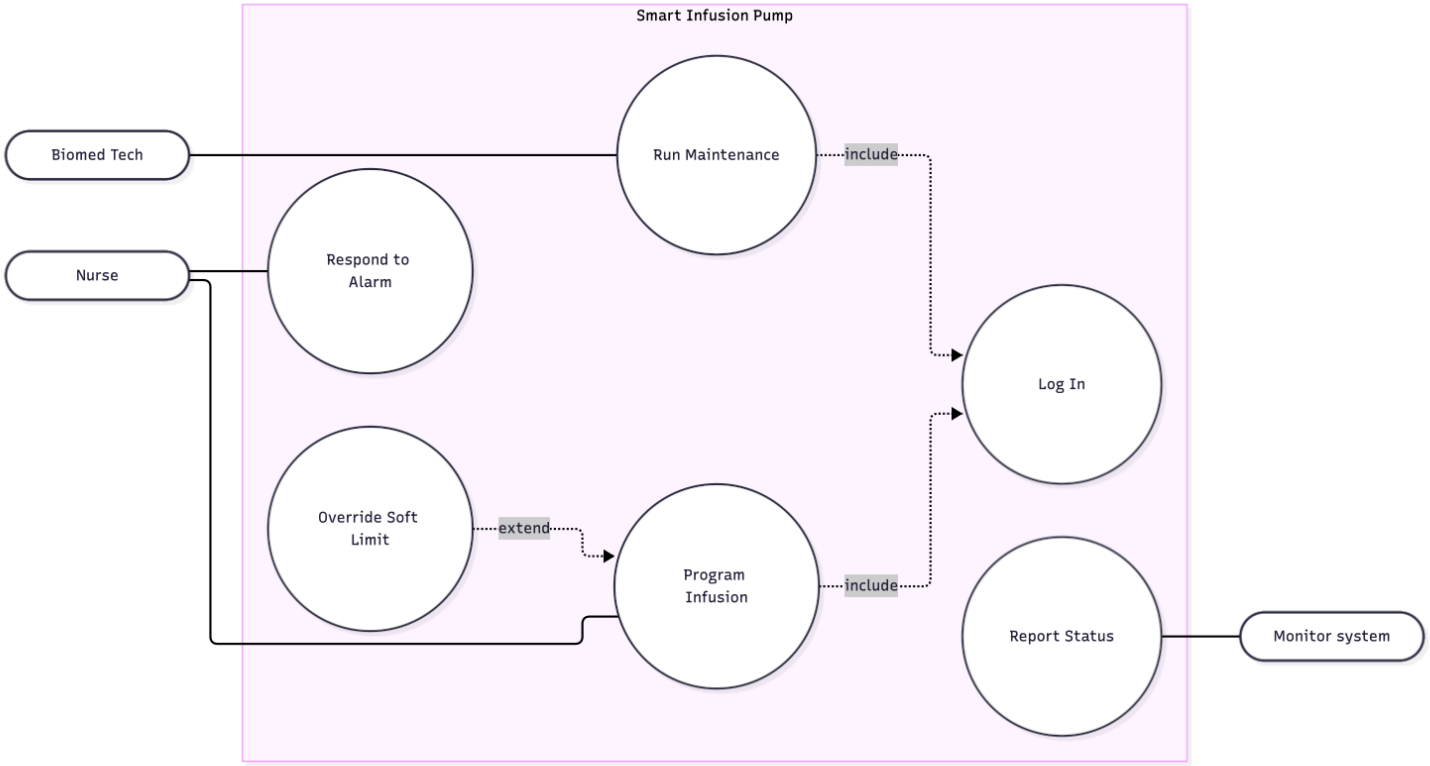


- 3 main components: Actors, System, Use Case
- Most often used for requirements and concepts
- Easiest form of modelling with the least rules
- Answers the question who will use our system, what will they use in the system, and sometimes how will they use the system
- **<<include>>**: A base use case always incorporates the behaviour of another use case, often to reuse common functionality.
- **<<extend>>**: An extending use case adds optional or conditional behaviour to a base use case at defined extension points.

UML: Use Case Diagram – Include vs. Extend

	«include»	«extend»
Meaning	The base use case **always** performs the included one	The extending use case **sometimes** adds behaviour to the base
Arrow direction	Base → included	Extension → base
Condition	None: mandatory	Yes: an extension point + condition
Can the base run alone?	No, it needs the included step	Yes, the extension is optional
Pump example	*Start Infusion* «include» *Log In*	*Override Soft Limit* «extend» *Program Infusion* [dose above soft limit]

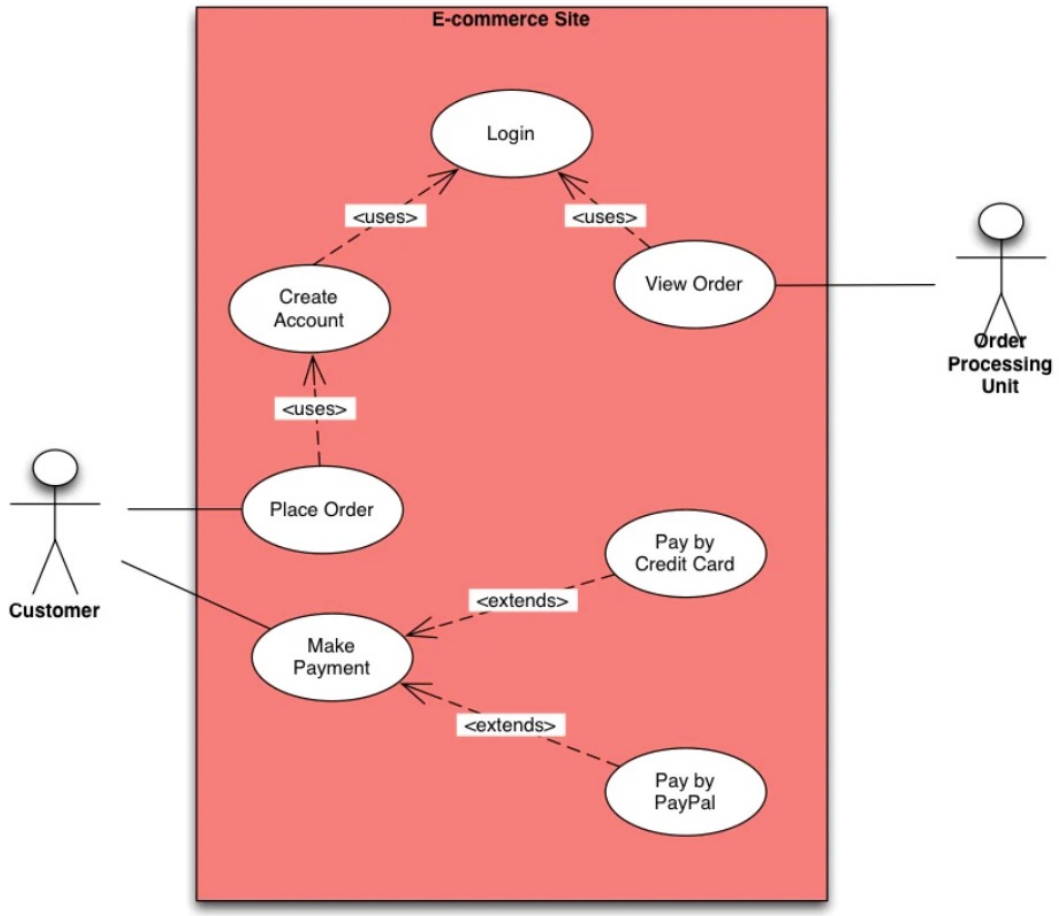
UML: Use Case Diagram – Pump Example



UML: Use Case Diagram – Exercises

In this example, *uses* is just equivalent to *includes*.

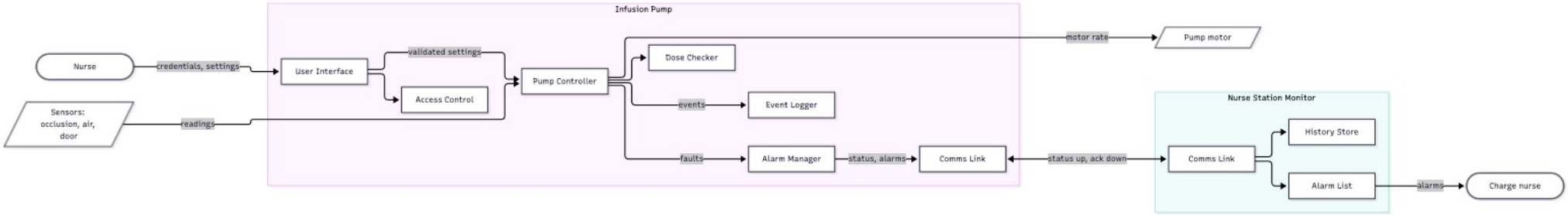
With the person next to you: **Identify** the actors, systems and use cases



High-Level Architecture Diagram

- A block/component diagram of the major parts and what flows between them.
- Shows: internal components, users/actors, hardware, the partner system, and labelled, directed connections.
- Does **not** show: classes, methods, module interfaces, the USES relation.
  - Those come in D2
- It is the **Component** column of your traceability table from last week!
  - Every component must trace to at least one requirement.

Pump – Monitor Architecture



Checklist Architecture Diagram

- Every actor from your use case diagram appears.
- Every hardware element (sensors, actuators, display, lock) appears as hardware, not software.
- Every arrow has a label (*what* is exchanged) and a direction.
- The boundary with the partner system is marked, and the messages crossing it are named.
- 6-12 components. If you have 30, you are doing D2 early.
- Each component traces to at least one requirement.

UML: Summary – Structure vs. Behaviour

Diagram	Purpose	When to Use	Key Elements
Class Diagram	Defines classes, attributes, methods, and relationships.	When designing the core structure of your system: data, logic, and relationships	Classes, attributes, methods, associations, inheritance, aggregation, compositions
Component/Deployment Diagrams	Show how software components are organized and deployed	When focusing on system architecture or physical deployment	Components, nodes, connections

UML: Summary – Structure vs. Behaviour

Diagram	Purpose	When to Use	Key Elements
Use Case Diagram	Captures system functionality from the user's perspective.	Early in design — to define *requirements* and understand *who interacts with what.*	Actors, use cases, include/extend relationships.
Activity Diagram	Models workflows, control flow, or data flow.	When describing *processes, algorithms,* or *business logic* step-by-step.	Start/end nodes, actions, decisions, forks, joins, edges.
State Machine Diagram	Models how an object changes state in response to events.	When designing *reactive systems*, e.g., embedded systems, UI states, control logic.	States, transitions, events, guard conditions.
Interaction Diagram	Category that includes sequence, communication, and timing diagrams.	When focusing on *communication patterns* between objects.	Entities and exchanged messages.
Sequence Diagram	Models interactions between components over time.	When describing *message flow* or *method calls* that implement a use case.	Lifelines, messages, activation bars, returns.

Think: The Pharmacy Addition

- The pharmacy now pushes drug library updates to every pump.
  - Add the new actor and the component(s) it talks to.
  - Label the flow and its direction.
  - List which requirements, stories and traceability rows change. (Expect: a new FR for accepting updates, an NFR for verifying them, a new stakeholder row.)

SFWRENG / MECHTRON 3K04 Software Development

Today: Domain Model and Activity Diagrams

McMaster University | Engineering Computing & Software

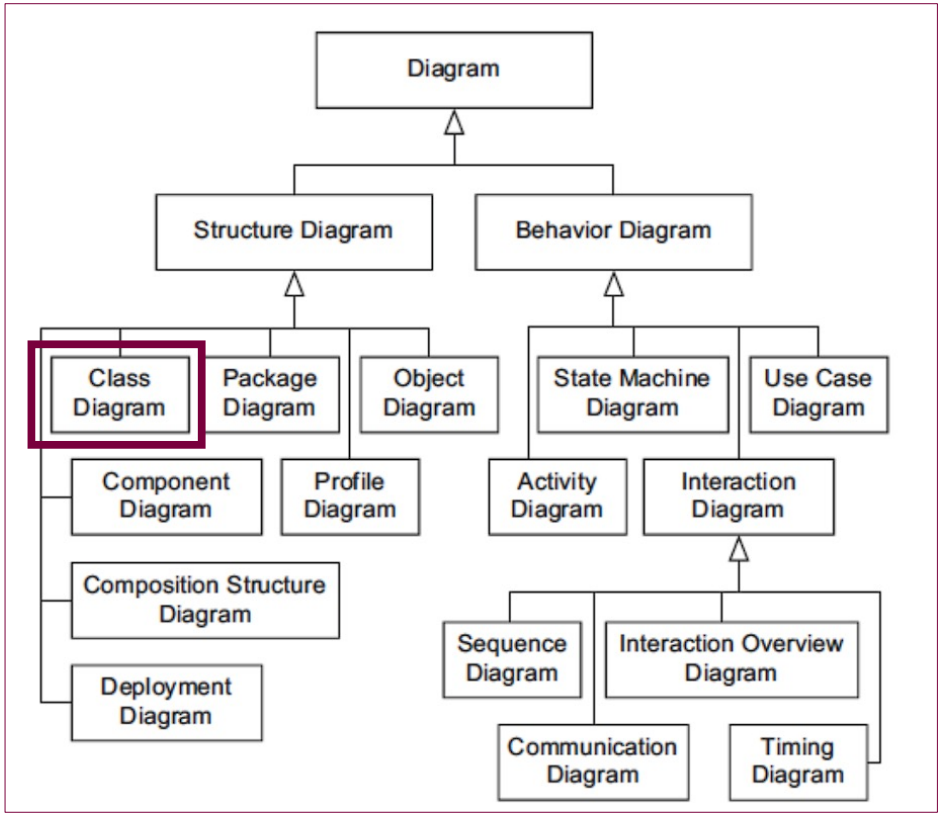
October 1st, 2026

Dr. Angela Zavaleta Bernuy

UML: Does it help modularity?

- Yes!
- Structure Diagrams
  - Recall: it defines the blueprint of the system
  - Architectural modularity: separation of systems into layers/components
- Behaviour Diagrams:
  - Recall: Communication between components
  - Interface modularity: shows how components communicate (communication contracts)

UML: Class Diagram

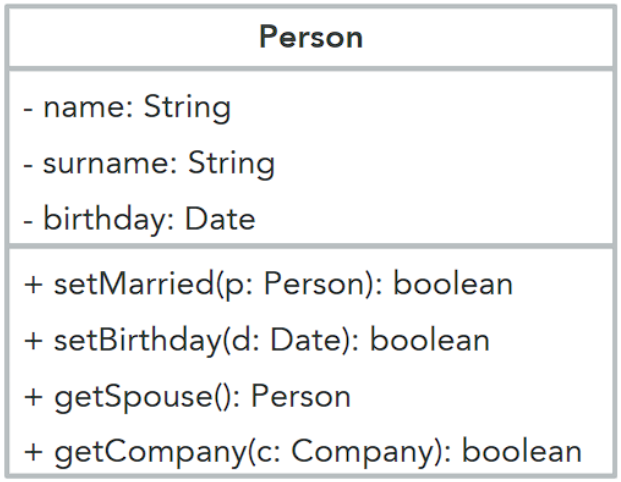


- Class diagram depicts the types of objects in a system and the static relationships between them.
- A class is generally made of 2 parts:
  - Name
  - Properties

Person
- name: String - surname: String - birthday: Date
+ setMarried(p: Person): boolean + setBirthday(d: Date): boolean + getSpouse(): Person + getCompany(c: Company): boolean

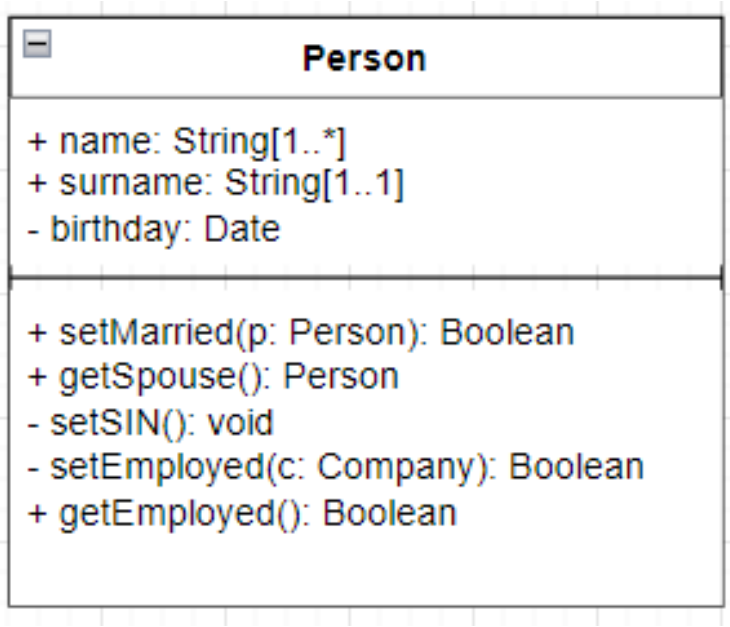
UML: Class Diagram

- Properties can be though of as fields in the class
  - Attributes
  - Methods
- Properties contain information such as private/public, multiplicities, return types and sometimes more.



UML: Class Diagram

- Visibility:
  - + public: can be accessed by any other class
  - - private: access only within the object permitted
  - # protected: access by objects within the same class and subclasses permitted
  - ~ friendly: access by objects within the same package
- Attribute:
  - Visibility name: type [multiplicity]
- Method:
  - Visibility name (parameters): return type

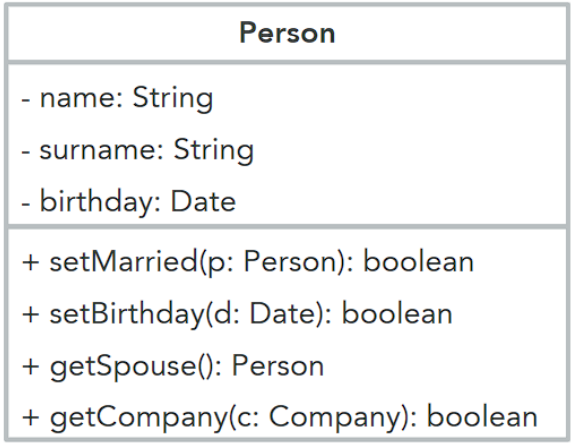


UML: Class Diagram

```
public class Person {
    private String name;
    private String surname;
    private Date birthday;

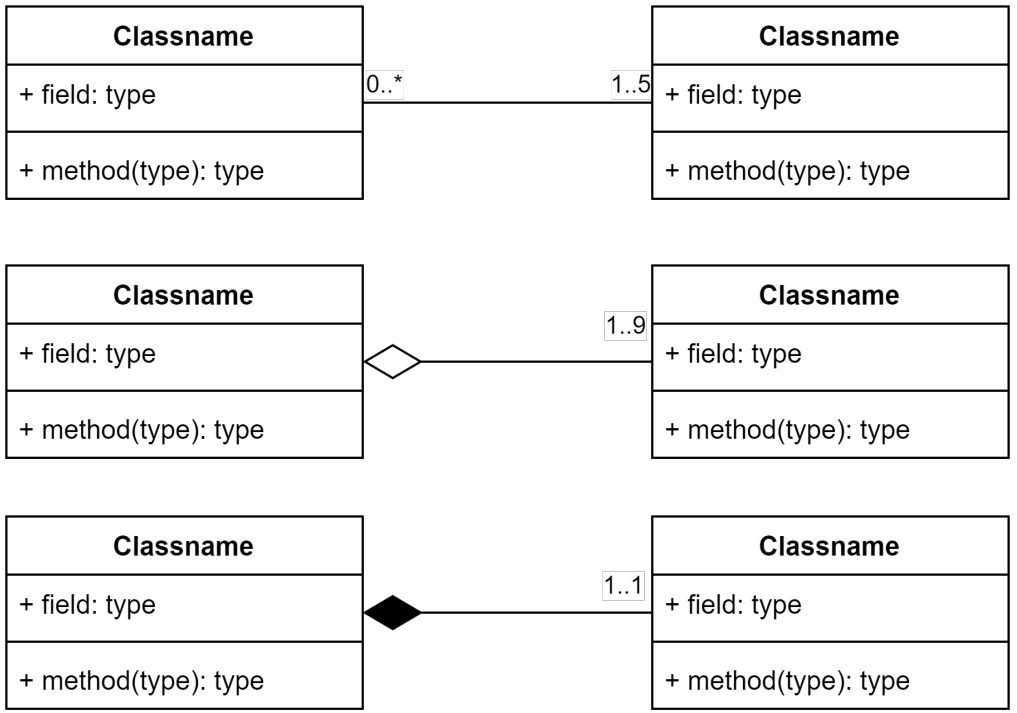
    public boolean getMarried (Person p) {...}
    public boolean setBirthday (Date d) {...}
}
```





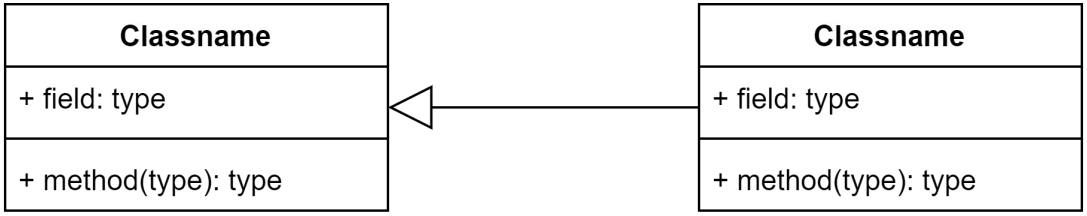
UML: Class Diagram – 5 Types of References

- Association
  - "*knows about*" / "*uses*"
- Aggregation
  - Special type of association
  - "*part of*" / "*has a*"
- Composition
  - "ownership"
  - If the class on the left is deleted, the class on the right is also deleted

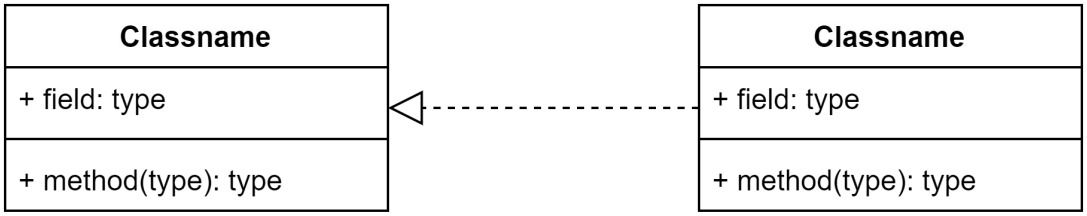


UML: Class Diagram – 5 Types of References

- Inheritance
  - "*is a*"



- Interface
  - "*uses*" / "*implements*"



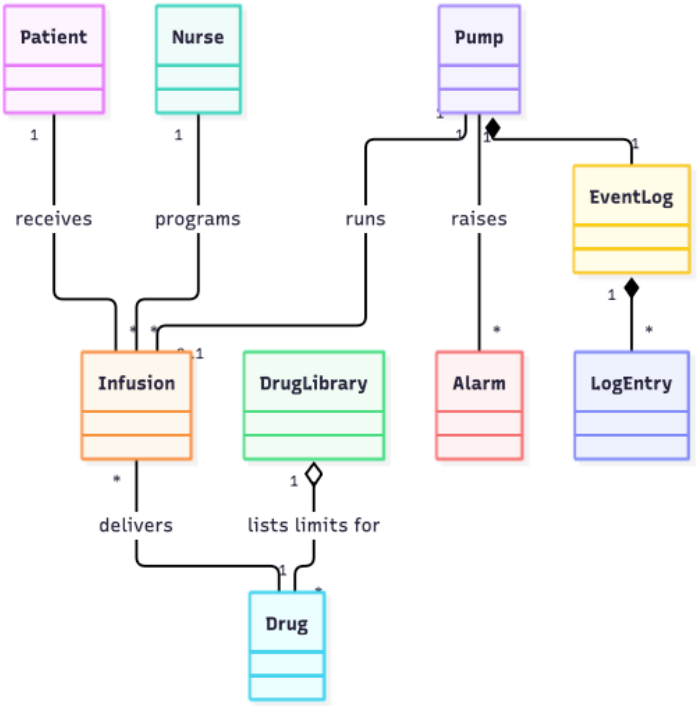
Domain Model vs. Design Class Diagram

	Domain (conceptual) model: D1	Design class diagram: D2
Purpose	Understand the problem's concepts	Plan the code
Contains	Real-world entities + relationships + multiplicities	Classes, attributes with types, methods, visibility
Example classes	Patient, Infusion, Drug	InfusionController, AlarmQueue
Visibility (+ – #)	Not needed	Needed

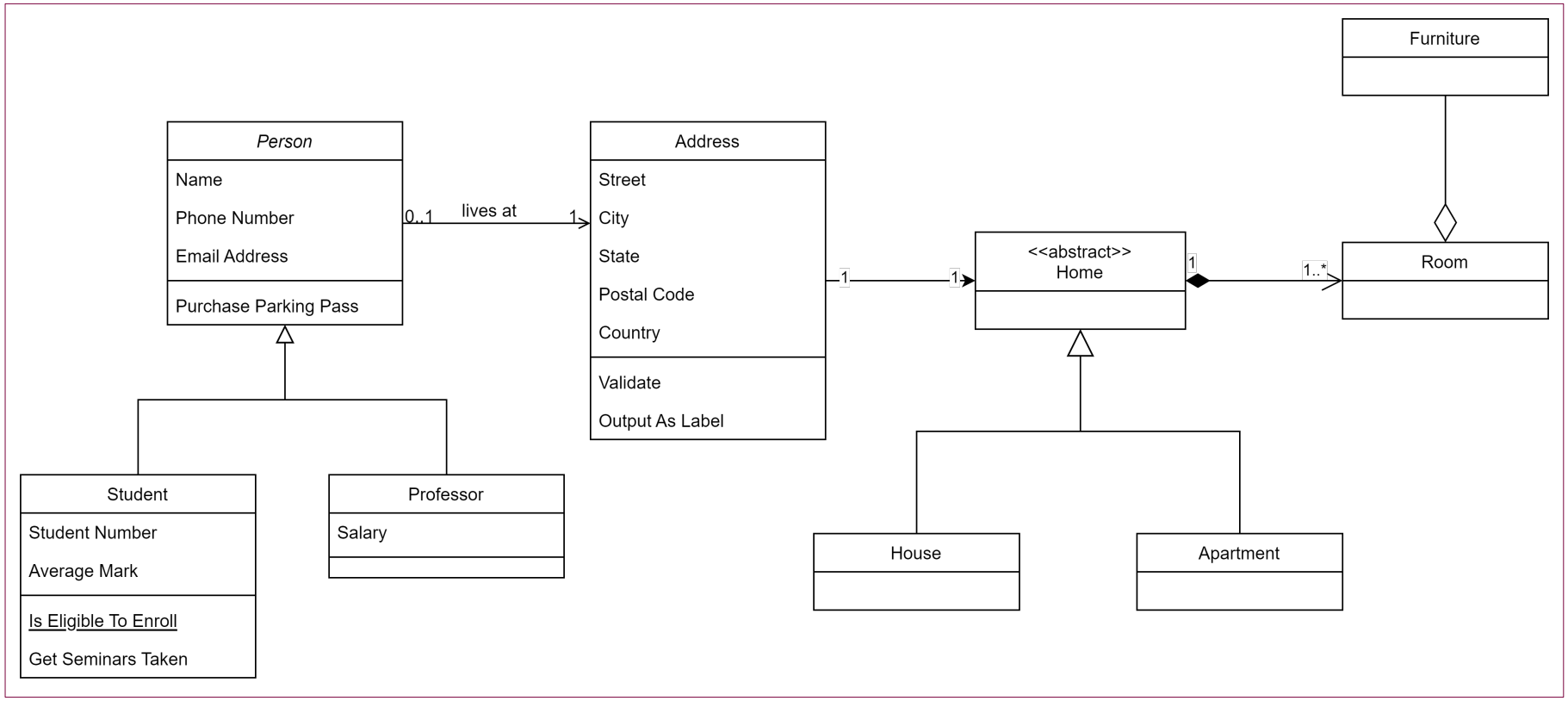
Finding Domain Classes

- Underline the nouns in your requirements and stories.
- Keep the ones the system must remember or reason about. Drop synonyms, UI words and vague words ("system", "data").
- Draw associations from verbs ("nurse *programs* infusion").
- Add multiplicities (1, 0..1, 1..*, *). Every association gets one at each end.
- Use composition only for real ownership: deleting the pump deletes its event log.

Pump Domain Model

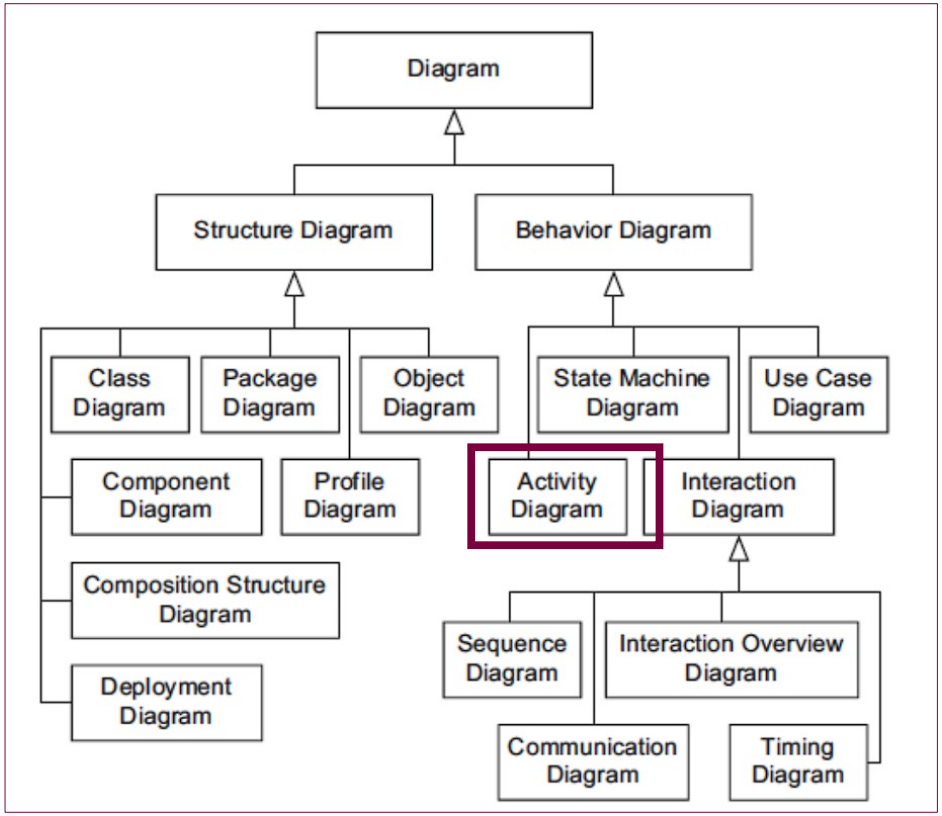


UML: Class Diagram – Exercise

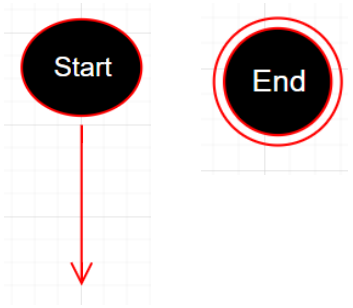


With the person next to you: **Identify** the relationships.

UML: Activity Diagrams



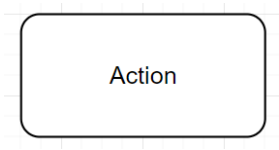
- Models the flow of control or workflow in a system or process
- Composed of a starting point and end point



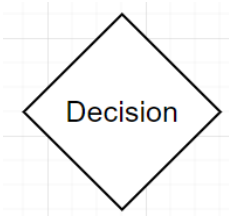
- 5 main components

UML: Activity Diagrams – 5 Main Components

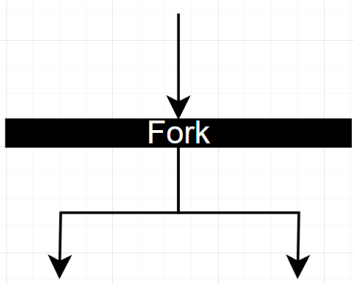
- Actions
  - Individual steps or operations



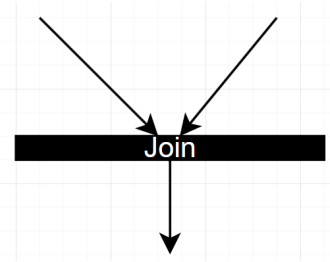
- Decisions
  - Branching points where the flow splits based on a condition



- Forks
  - Parallel execution: a single flow splits into multiple simultaneous paths

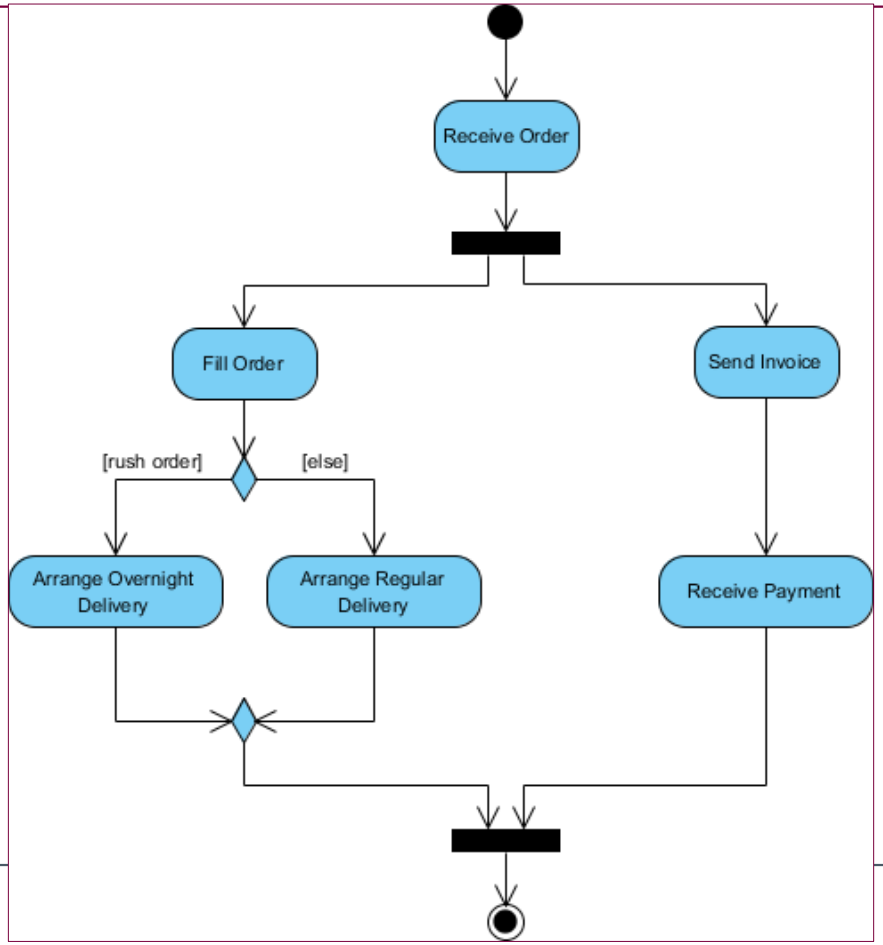


- Joins
  - Opposite of forks
  - Multiple flows merge back into one

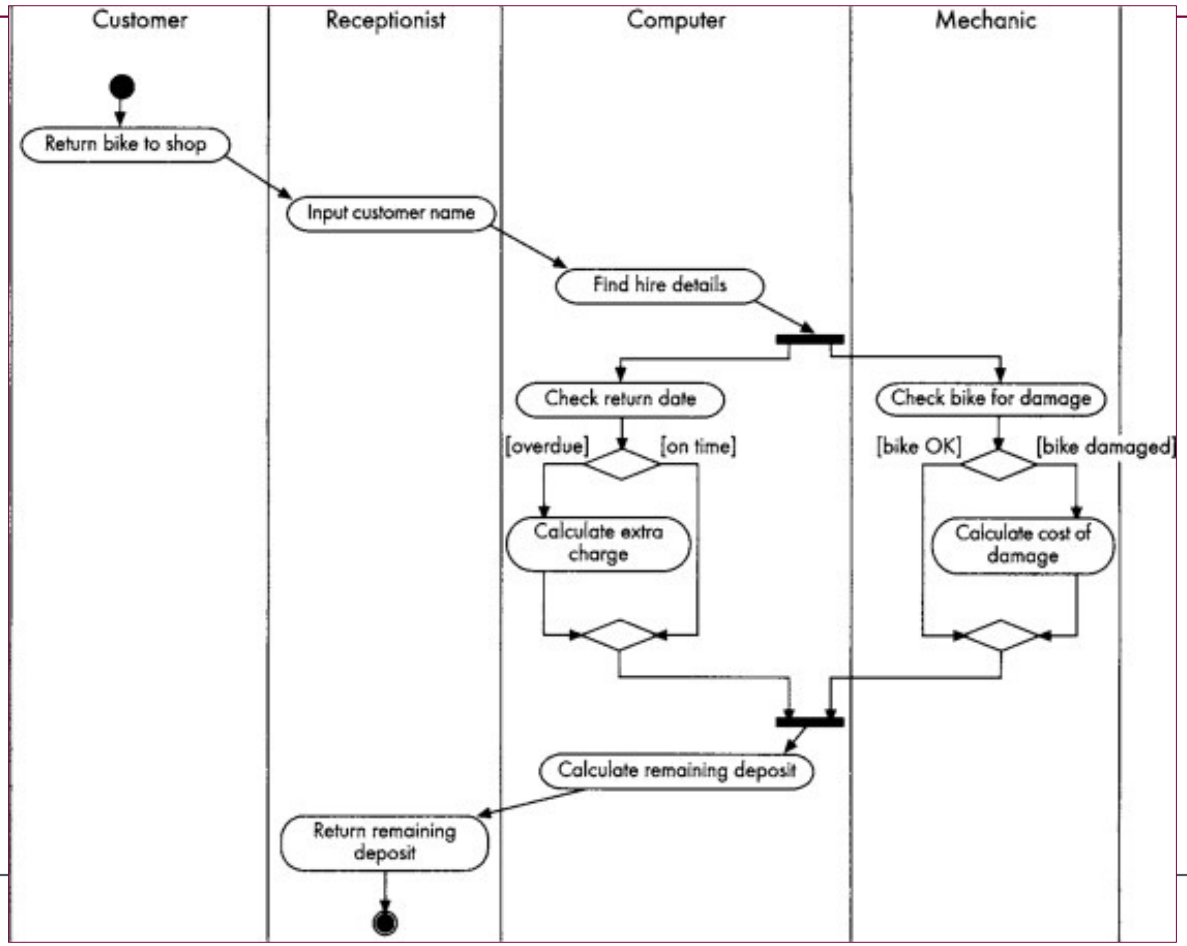


- Edges
  - Arrows that connect actions, decisions, forks, and joins

UML: Activity Diagram – Example



UML: Activity Diagram - Example



Activity Diagrams for Scenarios

- One activity diagram = one scenario from start to end (e.g. "Start an infusion").
- Swimlanes (partitions) show who does each action: Nurse / Pump / Monitor.
- Decision nodes carry guards in brackets: [dose ≤ soft limit], [else]. Guards on one decision must not overlap and must cover every case (disjoint + complete).
- Fork/join only for real parallel work (pump infuses *while* it reports status).
- Every path ends at a final node, including the error paths.

Normal vs. Off-Normal Paths

- The normal path is the story's happy path.
- Off-normal paths: hardware fault, invalid input, timeout, communication lost, power loss.
- D1 wants both.
- Each off-normal branch should match an acceptance criterion and an FR (fault detection, alarms)

Picking at Least 4 Scenarios

#	Scenario	Off-normal paths to include
1	Start an infusion	Wrong login; dose above hard limit; soft-limit override
2	Occlusion during an infusion	Nurse does not respond in 2 min → escalate
3	Communication lost with the monitor	Pump keeps working and alarming locally
4	Maintenance / self-test	Self-test fails → Fault mode

SFWRENG / MECHTRON 3K04 Software Development

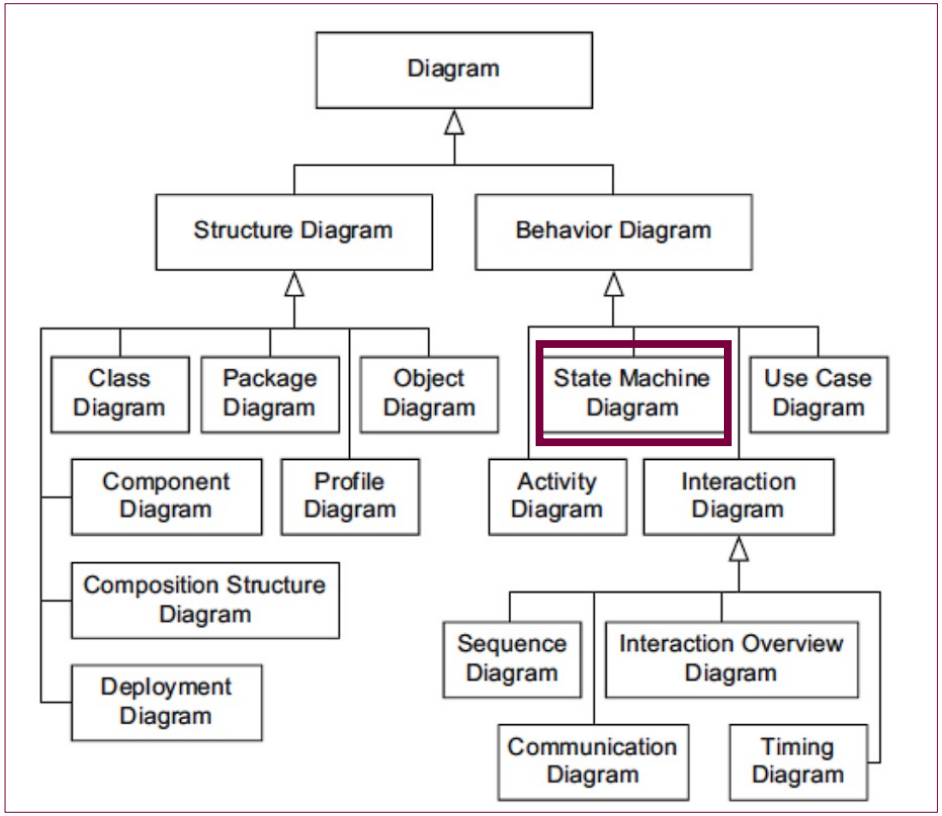
Today: State Machines and Consistency

McMaster University
Engineering
Computing & Software

October 2nd, 2026

Dr. Angela Zavaleta Bernuy

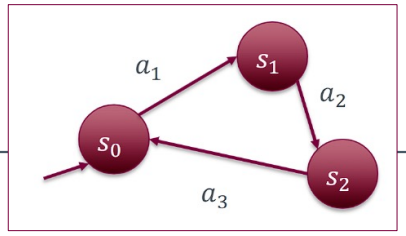
UML: State Machine Diagrams



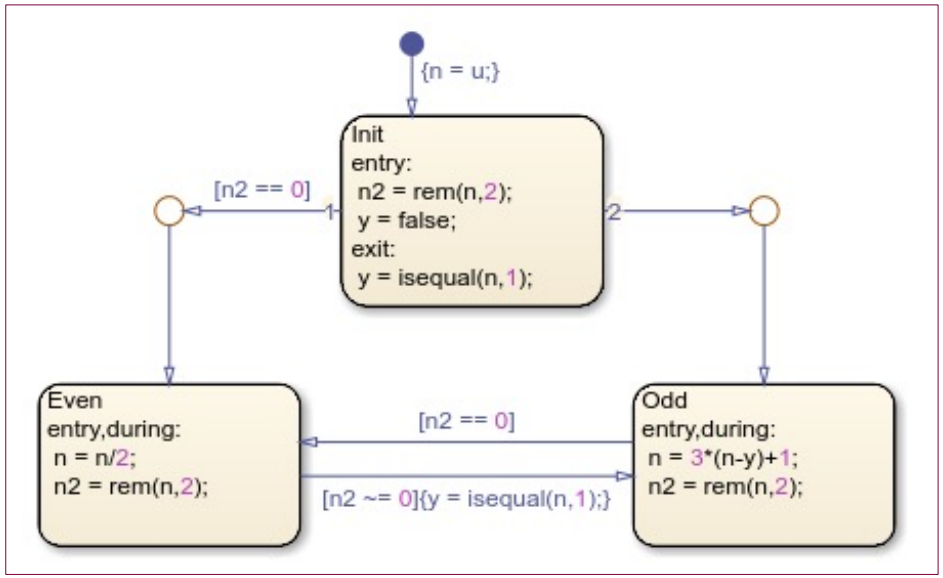
- Models how an object or system changes over time.
- Also called Finite State Machines (FSMs)
- Typically composed of a tuple <S, A, T>
  - S: a finite set of States
  - A: a finite set of Actions
  - T: a transition relations where

$$T \subseteq S \times A \times S \text{ or}$$
$$T : S \times A \rightarrow S$$

- Typically, it will have one initial state where everything starts.



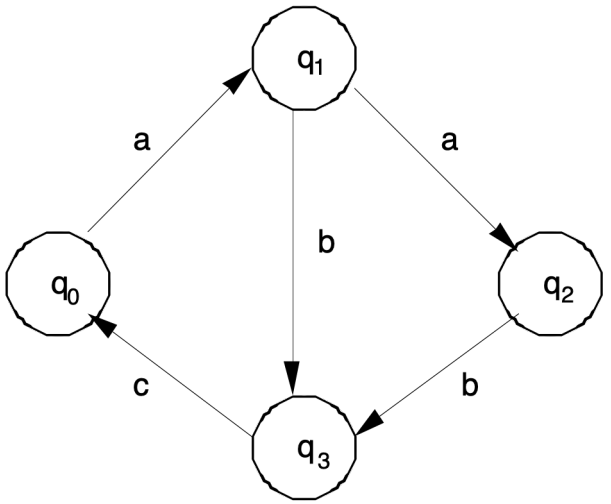
UML: State Machine Diagrams – Simulink Example



- The initial state in the top state
- Every state has actions to execute
- Transitions can have timing conditions or other Boolean conditions
- Transitions can also have actions to complete during transition (not common or recommended for beginners)

Finite State Machines (FSMs)

- Can specify control flow aspects, not just data flow
- They are defined as:
  - A finite set of states Q
  - A finite set of inputs I
  - A transition function d: Q x I -> Q telling us how the system moves from one state to another when it sees an input
- The diagram is a State Transition Diagram, showing the possible states and how inputs a, b, c trigger changes.



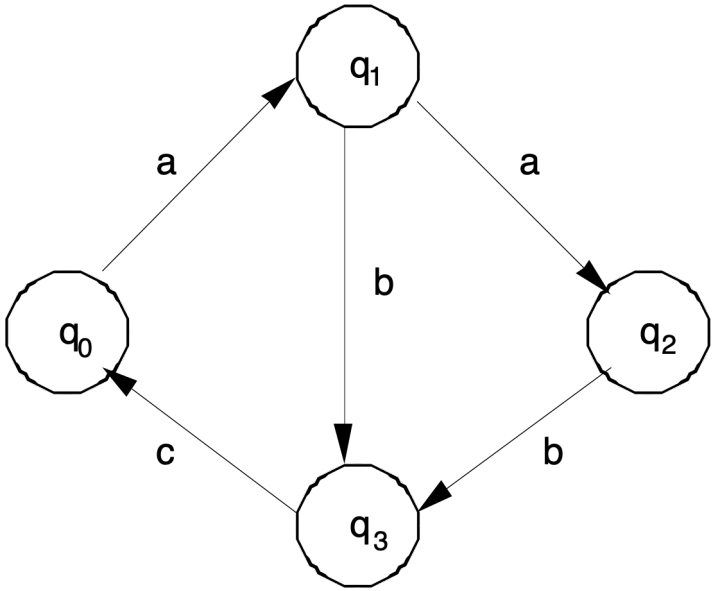
Finite State Machines (FSMs)

State Transition Table

	q_0	q_1	q_2	q_3
a	q_1	q_2	-	-
b	-	q_3	q_3	-
c	-	-	-	q_0

Cannot occur simultaneously

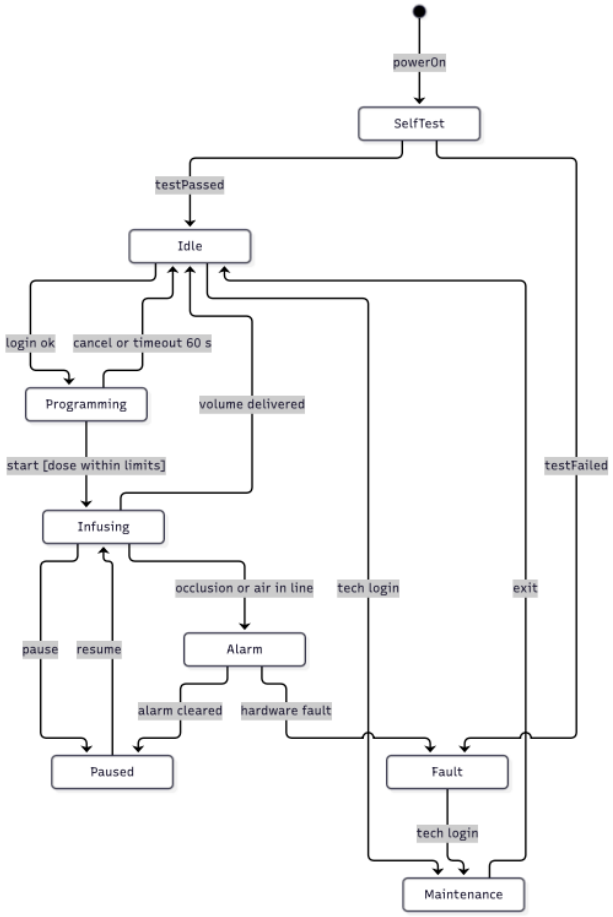
State Transition Diagram



UML State Machine Notation

- State: a condition the system stays in (rounded box). Name it with a noun or adjective: *Infusing*, *Paused*.
- Transition label: event [guard] / action, e.g. occlusionDetected [infusing] / stopMotor.
- Initial (filled dot) and final (bullseye) pseudo-states.
- Composite state: a state containing sub-states (e.g. *Operational* contains *Idle, Programming, Infusing, Paused*).
- Entry / exit actions inside a state: entry / soundAlarm.
- Every event the system can receive should be handled, or explicitly ignored, in every state. This is the completeness property again.

UML State Machine Notation



Invalid and Fault Events

- Invalid event = an event that makes no sense in the current state (*resume* while *Idle*). Choose one: ignore it, reject with a message, or log it. Write the choice down as a requirement.
- Fault event = hardware or software failure. It must lead to a *safe state* from any state. Draw it once from the composite state rather than 10 times.
- A state transition table (W6P1 s3) is the best way to check you have not missed a cell: states × events, every cell filled or marked "

Ready for the demo?

Keeping the Views Consistent

Check	Views involved
Every story traces to a requirement **and** appears in a use case	Stories, FRs, use case diagram
Every scenario in just shown has an activity diagram	Scenarios, activity diagrams
State names and events match the modes and events in the FRs	State machines, FRs
Every actor, external system and hardware item appears on the architecture diagram	Use case, activity, architecture
Domain classes use the same names as the requirements	Domain model, FRs, stories
Every traceability row is complete	All

The Three-View Walk

- TAs will pick one item and ask you to move between views. Practise it live with the pump:
- Requirement: FR-ALM-02, "The pump shall stop and alarm on occlusion."
- Story: US-ALM-01, the nurse at the bedside, and its Gherkin criteria.
- Use case: *Respond to Alarm*.
- Activity: scenario 2, occlusion during infusion.
- State machine: *Infusing* → *Alarm* on occlusion.
- Architecture: Sensors → Pump Controller → Alarm Manager → Comms Link → Monitor.

Demo Prep Checklist

- Every member can explain any requirement, story and diagram (TAs choose who answers).
- Requirements document is about 95% done before the lab demo.
- ID scheme is stable; no renumbering after the demo.
- Assumptions log includes ethics and accessibility decisions.
- Rehearse the three-view walk on 3 random requirements.
- Rehearse one on-the-spot change as a team.

Questions?